



2- Notificaciones de Cambio de Propiedad

Interfase INotifyPropertyChanged

Permite notificar a un cliente que el valor de una propiedad cambió. Se usa generalmente en clientes enlazados para notificarlos que una propiedad fue modificada. Para controlar estas notificaciones entre clientes y fuentes de datos enlazados se podría hacer de dos maneras:

- Implementar una clase con la interfase INotifyPropertyChanged
- Generar eventos de cambios para cada propiedad

La primera opción es la recomendada.

Supongamos el siguiente ejemplo. Tenemos una clase Persona con dos propiedades Nombre y Edad.

```
public class Persona
{
    private string _Nombre;
    public string Nombre
    {
        get { return _Nombre; }
        set { _Nombre =value;}
    }

    int _Edad;
    public int Edad
    {
        get { return _Edad; }
        set { _Edad=value;}
    }

    public Persona( ) {}
    public Persona(string pNombre, int pEdad)
    {
        Nombre = pNombre ;
        Edad = pEdad;
    }
}
```

Y la siguiente interfase de usuario que muestra los datos de una instancia de la clase Persona y permite modificar la edad cada vez que se presiona el botón de Cumpleaños:

```
<Window x:Class="Notificaciones.Window1"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
Title="Window1" Height="300" Width="300" Loaded="Window_Loaded">
<Grid VerticalAlignment="Top" HorizontalAlignment="Left" Width="250" Height="100">
    <Grid.ColumnDefinitions >
        <ColumnDefinition />
        <ColumnDefinition />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions >
        <RowDefinition />
        <RowDefinition />
        <RowDefinition />
    </Grid.RowDefinitions>
```



```
</Grid.RowDefinitions>
<TextBlock Grid.Row="0" Grid.Column="0" >Nombre:</TextBlock>
<TextBox Grid.Row="0" Grid.Column="1" Height="23" Name="txtNombre" />
<TextBlock Grid.Row="1" Grid.Column="0">Edad:</TextBlock>
<TextBox Name="txtEdad" Grid.Row="1" Grid.Column="1" Height="23"/>
<Button Grid.Row="2" Grid.ColumnSpan="2" Name="btnCumple"
        Click="btnCumple_Click">Cumpleaños</Button>
</Grid>
</Window>
```

```
public partial class Window1 : Window
{
    Persona oPersona;
    public Window1()
    {
        InitializeComponent();
    }

    private void Window_Loaded(object sender, RoutedEventArgs e)
    {
        oPersona = new Persona("Juan", 10);
        txtNombre.Text = oPersona.Nombre;
        txtEdad.Text = oPersona.Edad.ToString();
    }

    private void btnCumple_Click(object sender, RoutedEventArgs e)
    {
        oPersona.Edad++;
        MessageBox.Show("Feliz Cumpleaños");
        // Actualizamos manualmente la propiedad
        txtEdad.Text = oPersona.Edad.ToString();
    }
}
```

Este ejemplo funciona perfectamente, pero como no tiene sincronizados los valores del oPersona con los TextBoxs estos tienen que actualizarse manualmente cada vez que se genera un cambio en alguno de los dos valores.

Una forma para que la UI lleve registro de los cambios, mejor que la del ejemplo anterior, es que el objeto avise cuando una propiedad cambia, por ejemplo, lanzando un evento. La forma correcta de hacer esto, es que el objeto implemente **INotifyPropertyChanged**.

```
using System.ComponentModel;
public class Persona: INotifyPropertyChanged
{
    .....
    #region Miembros de INotifyPropertyChanged
        public event PropertyChangedEventHandler PropertyChanged;
    #endregion
}
```

De esta manera deberíamos entonces agregar a la clase Persona las siguientes líneas. Estas líneas agregan un evento a la clase llamado **PropertyChanged** y el mismo es disparado a



través del método *Notificar*, que es invocado por los Set de las propiedades, reflejando el cambio.

```
public class Persona: INotifyPropertyChanged
{
    private string _Nombre;
    public string Nombre
    {
        get { return _Nombre; }
        set {
            _Nombre = value;
            Notificar ("Nombre");
        }
    }

    int _Edad;
    public int Edad
    {
        get { return _Edad; }
        set {
            _Edad=value;
            Notificar ("Edad");
        }
    }

    public Persona( ) {}
    public Persona(string pNombre, int pEdad)
    {
        Nombre = pNombre ;
        Edad = pEdad;
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void Notificar (string NombrePropiedad)
    {
        if (this.PropertyChanged != null)
        {
            PropertyChanged(this, new
            PropertyChangedEventArgs(NombrePropiedad));
        }
    }
}
```

El código de la ventana quedaría de la siguiente manera. Atajamos el evento disparado por la clase Persona y además necesitaríamos también un mecanismo que detecte los cambios en la interfase gráfica, y los propague hacia el objeto, de manera que la sincronización sea en ambos sentidos. Para ellos atajamos el evento *TextChanged* de los TextBox y modificamos el valor de las propiedades. De esta manera mantenemos ambos valores sincronizados permanentemente.



```
public partial class Window1 : Window
{
    Persona oPersona;
    public Window1()
    {
        InitializeComponent();
    }

    private void Window_Loaded(object sender, RoutedEventArgs e)
    {
        oPersona = new Persona("Juan", 10);
        txtNombre.Text = oPersona.Nombre;
        txtEdad.Text = oPersona.Edad.ToString();
        oPersona.PropertyChanged += new
System.ComponentModel.PropertyChangedEventHandler(
Persona_PropertyChanged);
    }

    private void btnCumple_Click(object sender, RoutedEventArgs e)
    {
        oPersona.Edad++;
        MessageBox.Show("Feliz Cumpleaños");
    }

    private void Persona_PropertyChanged(object sender,
    System.ComponentModel.PropertyChangedEventArgs e)
    {
        switch( e.PropertyName )
        {
            case "Nombre":
                txtNombre.Text = oPersona.Nombre;
                break;
            case "Edad":
                txtEdad.Text = oPersona.Edad.ToString();
                break;
        }
    }

    private void txtNombre_TextChanged(object sender, TextChangedEventArgs e)
    {
        oPersona.Nombre = txtNombre.Text;
    }

    private void txtEdad_TextChanged(object sender, TextChangedEventArgs e)
    {
        oPersona.Edad = System.Convert.ToInt32(txtEdad.Text);
    }
}
```

Sin importar entonces en donde se de el cambio (objeto ,UI), ambos elementos se mantienen sincronizados.

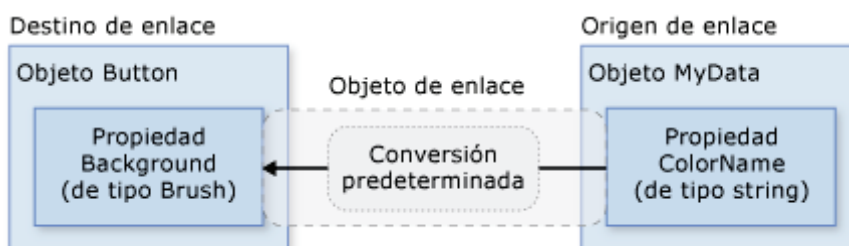
Claramente, la cantidad de código a escribir puede llegar a ser importante si la cantidad de objetos, propiedades de los objetos y eventos aumenta. Además, este tipo de tareas parecen

bastante repetitivas. Por eso es que WPF las abstrae en funcionalidades del *framework*, dándole el nombre de *Data Binding*.

3- Convertir Datos

ValueConverters

Si se desea enlazar dos propiedades de diferente tipo se deberá usar un *ValueConverter*. Los *ValueConverter* convierten los valores del tipo de dato de la fuente al tipo de dato del destino o viceversa. Para generar clases convertidora personalizadas es necesario usar la interfase *IValueConverter*



Interfase IValueConverter

Esta interfase permite generar clases que apliquen código de conversión de datos que pueden usarse en enlaces de datos. Para ellos cree una nueva clase e implemente la interfase *IValueConverter*. Esta interfase contiene dos métodos *Convert* y *ConvertBack*. Esta clase podrá convertir los datos de un tipo de dato a otro o modificar el aspecto de la presentación. Ambos métodos están asociados a culturas. Si el uso de culturas no es necesario en la aplicación con la cual se está trabajando este parámetro puede ignorarse.

Por ejemplo si necesitamos enlazar un valor verdadero o falso a la propiedad *Visibility* debemos convertir el dato, ya que esta propiedad es una enumeración que contiene tres valores: Visible, Collapsed o Hidden.

Como primer paso escribiremos una clase que implemente a la interfase *IValueConverter* y que implemente el método *Convert* de manera de generar la conversión necesaria. En este ejemplo no se está permitiendo la conversión inversa.

```
using System.Windows.Data;
using System.Globalization;
using System.Windows;
```

```
public class ConversionBooleanToVisibility: IValueConverter
{
    public object Convert(object value, Type targetType, object parameter,
        CultureInfo culture)
    {
        if (value is Boolean)
        {
            return ((bool)value) ? Visibility.Visible : Visibility.Collapsed;
        }
        return value;
    }

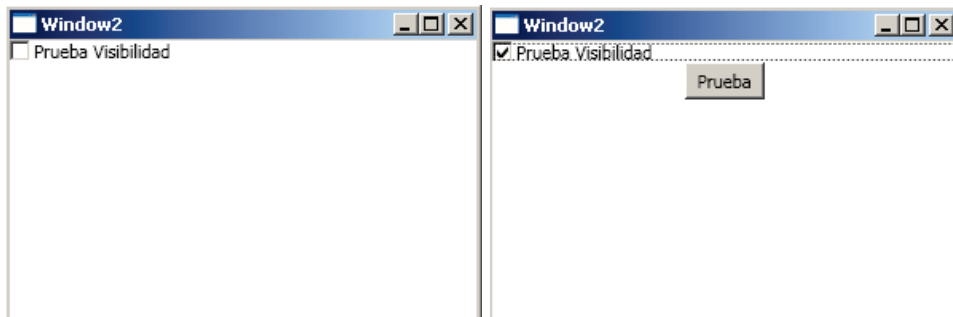
    public object ConvertBack(object value, Type targetType, object parameter,
        CultureInfo culture)
```



```
{  
    throw new NotImplementedException();  
}  
}
```

Luego crearemos un recurso que nos permita relacionar esta ventana con la clase. Al momento del enlace usando el atributo Converter podemos generar la llamada a nuestro convertidor.

```
<Window x:Class="WpfCSharp.Window2"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    Title="Window2" Height="200" Width="300"  
    xmlns:svc="clr-namespace:WpfCSharp" >  
    <StackPanel>  
        <StackPanel.Resources>  
            <svc:ConversionBooleanToVisibility x:Key="boolToVisible" />  
        </StackPanel.Resources>  
  
        <CheckBox x:Name="chkPrueba" Content="Prueba Visibilidad" />  
        <StackPanel x:Name="Detalle"  
            Visibility="{Binding IsChecked, ElementName=chkPrueba,  
                Converter={StaticResource boolToVisible}}">  
            <Button Height="23" Width="50" Content="Prueba"></Button>  
        </StackPanel>  
    </StackPanel>  
</Window>
```



En el siguiente ejemplo se crea una clase para controlar la validación y formateo de datos de tipo fecha. Al escribir una fecha sobre el *TextBox*, cuando reconoce que lo escrito tiene un formato tipo fecha la devuelve en formato dd/mm/aaaa en el *TextBlock*

```
[ValueConversion(typeof(DateTime), typeof(String))]  
public class ConvierteFecha : IValueConverter  
{  
    public object Convert(object value, Type targetType, object parameter, CultureInfo  
        culture)  
    {  
        DateTime fecha;  
        if (value != null && DateTime.TryParse(value.ToString(), out fecha))  
        {  
            return fecha.ToString("dd/MM/yyyy");  
        }  
    }  
}
```

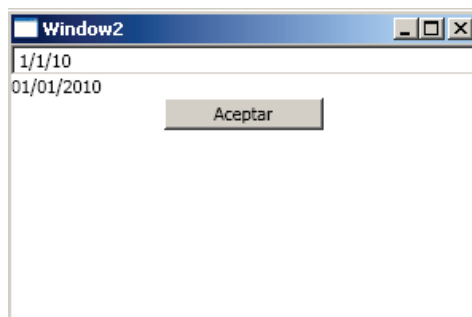
```

    }
    else
        return "";
}

public object ConvertBack(object value, Type targetType, object parameter, CultureInfo
culture)
{
    string strValue = (string)value;
    DateTime fecha;
    if (DateTime.TryParse(strValue, out fecha))
        return fecha;
    else
        throw new NotImplementedException();
}
}

<Window.Resources >
    <svc:ConvierteFecha x:Key="ConvFecha" />
</Window.Resources>
<StackPanel>
    <TextBox Name="txtFecha" />
    <TextBlock Name="tbFecha" Text="{Binding ElementName=txtFecha,
Path=Text, Converter={StaticResource ConvFecha}}" />
    <Button Height="20" Width="100" Content="Aceptar"></Button>
</StackPanel>

```



El atributo **ValueConversion** permite especificar el tipo de dato de entrada y el de salida, es útil para documentar el tipo de conversión que estamos realizando, pero no es obligatorio su uso para que la conversión funcione correctamente.

4- Validar Datos

ValidationRules

La clase *Binding* cuenta con una propiedad llamada *ValidationRules*, que puede almacenar varias clases derivadas de *ValidationRule*. Todas esas reglas pueden contener cierta lógica que intenta comprobar si el valor enlazado es válido. Se pueden crear reglas personalizadas creando clases que deriven de *ValidationRule* o usar la clase *ExceptionValidationRule* que invalida el dato si existen excepciones durante el enlace.



Clase Binding.ValidationRule

Permite crear reglas personalizadas de manera de poder validar el ingreso de datos enlazados.

En el siguiente ejemplo se creará una clase que valida el rango de edad aceptado en una aplicación. Valida que solo se ingresen números y que el rango esté entre dos valores que pueden establecerse a través de propiedades.

```
public class ReglaEdad: ValidationRule
{
    private int _minEdad;
    public int MinEdad
    {
        get { return _minEdad; }
        set { _minEdad = value; }
    }

    private int _maxEdad;
    public int MaxEdad
    {
        get { return _maxEdad; }
        set { _maxEdad = value; }
    }

    public override ValidationResult Validate(object value, CultureInfo cultureInfo)
    {
        int iEdad = 0;
        try
        {
            if (((string)value).Length > 0)
                iEdad = Int32.Parse((String)value);
        }
        catch (Exception e)
        {
            return new ValidationResult(false, "Carácter incorrecto o " + e
                .Message);
        }

        if ((iEdad < MinEdad) || (iEdad > MaxEdad))
        {
            return new ValidationResult(false,
                "Por favor ingrese la edad en el rango: " + MinEdad + " - " +
                MaxEdad + ".");
        }
        else
        {
            return new ValidationResult(true, null);
        }
    }
}
```

Se crea también una clase llamada GenerarPersonas que devuelve un objeto Persona (definido en el punto 2 de este mismo capítulo). La llamada a este método generará la fuente de datos para el enlace.



```
public class GenerarPersonas
{
    public Persona TraerPersona()
    {
        Persona oPersona = new Persona();
        oPersona.Nombre = "Juan";
        oPersona.Edad = 1;
        return oPersona;
    }
}
```

Enlazamos entonces la propiedad Edad del objeto Persona a un TextBox llamado tbEdad. También le asociamos la regla ReglaEdad y le damos valor a las propiedades MaxEdad y MinEdad de manera que establecer el rango.

```
<TextBox Name="tbEdad" DataContext="{StaticResource Datos}"
Width="100">
    <TextBox.Text>
        <Binding Path="Edad" UpdateSourceTrigger="PropertyChanged" >
            <Binding.ValidationRules>
                <svc:ReglaEdad MinEdad="1" MaxEdad="80" />
            </Binding.ValidationRules>
        </Binding>
    </TextBox.Text>
</TextBox>
```

Creamos una plantilla y su estilo asociado para poder marcar los TextBox con error mostrando un punto rojo a la derecha del mismo y el mensaje de error en un Tooltip.

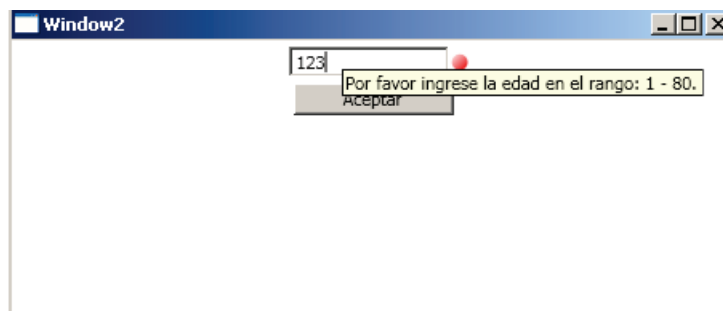
```
<Window.Resources>
    <ObjectDataProvider x:Key="Datos" ObjectType="{x:Type
        svc:GenerarPersonas}" MethodName="TraerPersona"></ObjectDataProvider>
    <ControlTemplate x:Key="TextBoxErrorTemplate">
        <DockPanel>
            <Ellipse DockPanel.Dock="Right" Margin="2,0" Width="10" Height="10">
                <Ellipse.Fill>
                    <LinearGradientBrush>
                        <GradientStop Color="#11FF1111" Offset="0" />
                        <GradientStop Color="#FFFF0000" Offset="1" />
                    </LinearGradientBrush>
                </Ellipse.Fill>
            </Ellipse>
            <AdornedElementPlaceholder />
        </DockPanel>
    </ControlTemplate>

    <Style x:Key="EstiloError" TargetType="TextBox">
        <Setter Property="Margin" Value="4,4,10,4" />
        <Setter Property="Validation.ErrorTemplate" Value="{StaticResource
            TextBoxErrorTemplate}" />
        <Style.Triggers>
            <Trigger Property="Validation.HasError" Value="True">
                <Setter Property="ToolTip">
                    <Setter.Value>
```



```
<Binding Path="(Validation.Errors)[0].ErrorContent"
RelativeSource="{x:Static RelativeSource.Self}" />
</Setter.Value>
</Setter>
</Trigger>
</Style.Triggers>
</Style>
</Window.Resources>

<StackPanel>
<TextBox Name="tbEdad" DataContext="{StaticResource Datos}"
Style="{StaticResource EstiloError}" Width="100">
<TextBox.Text>
<Binding Path="Edad" UpdateSourceTrigger="PropertyChanged" >
<Binding.ValidationRules>
<svc:ReglaEdad MinEdad="1" MaxEdad="80" />
</Binding.ValidationRules>
</Binding>
</TextBox.Text>
</TextBox>
<Button Height="20" Width="100" Content="Aceptar"></Button>
</StackPanel>
```



IDataErrorInfo

Con la llegada de Microsoft .NET Framework 3.5, la compatibilidad de WPF para la validación de entradas ha mejorado significativamente. *ValidationRule* es de utilidad para las validaciones sencillas, pero las aplicaciones del mundo real se enfrentan a la complejidad de los datos y las reglas de negocio. La codificación de reglas de negocio en objetos *ValidationRule* no sólo vincula dicho código a la plataforma WPF, sino que además no permite que haya lógica de negocio ahí donde debe existir: en los objetos de negocios.

Muchas aplicaciones tienen una capa de negocio, en la que la complejidad del procesamiento de reglas de negocio se incluye en un conjunto de objetos de negocios. Al compilar en Microsoft .NET Framework 3.5, puede usar la interfase *IDataErrorInfo* para que WPF pregunte a los objetos de negocios si están en un estado válido o no. De esta forma, se elimina la necesidad de agregar a los objetos una lógica de negocio independiente desde la capa de negocio, y permite crear objetos de negocios independientes de la plataforma de la interfase de usuario. Dado que la interfase *IDataErrorInfo* existe desde hace años, se facilita en gran medida el uso repetido de objetos de negocios de una aplicación heredada de Windows Forms o ASP.NET.

IDataErrorInfo implementa 2 propiedades:

- Error: Devuelve el mensaje de error indicando lo que no es correcto en el objeto



- Item: Devuelve el mensaje de error para la propiedad

Para el ejemplo tomaremos la misma clase *Persona* del ejemplo anterior y en ella implementaremos la interfase *IDataErrorInfo*.

```
using System.ComponentModel;
public class Persona : IDataErrorInfo
{
    private string _Nombre;
    public string Nombre
    {
        get { return _Nombre; }
        set { _Nombre = value; }
    }

    int _Edad;
    public int Edad
    {
        get { return _Edad; }
        set { _Edad = value; }
    }

    public Persona() { }
    public Persona(string pNombre, int pEdad)
    {
        Nombre = pNombre;
        Edad = pEdad;
    }

    public string Error
    {
        get { throw new NotImplementedException(); }
    }

    public string this[string columnName]
    {
        get
        {
            string result = null;
            if (columnName == "Nombre")
            {
                if (String.IsNullOrEmpty(Nombre))
                    result = "Debe ingresar el nombre";
                else if (Nombre.Length < 3)
                    result = "El nombre debe tener al menos 3 caracteres";
            }
            else if (columnName == "Edad")
            {
                // Verifica rango
                if ((Edad < 1) || (Edad > 80))
                {
                    result="El edad debe estar entre 1 y 80";
                }
            }
        }
    }
}
```



```

    }
    return result;
}
}
}

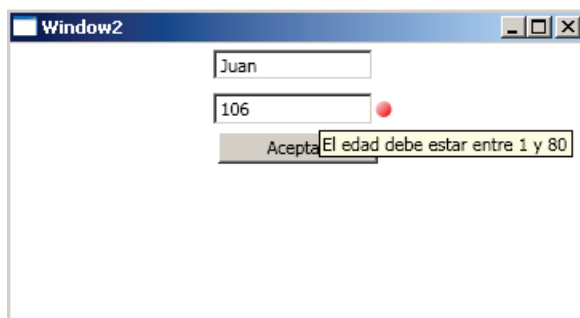
```

Teniendo en cuenta las mismas fuentes de datos, plantilla y estilo del ejemplo anterior modificamos el código para que valide usando las propiedades de esta interfase.

```

<StackPanel>
    <TextBox Name="txtNombre" DataContext="{StaticResource Datos}"
        Style="{StaticResource EstiloError}" Width="100">
        <TextBox.Text>
            <Binding Path="Nombre" UpdateSourceTrigger="PropertyChanged">
                <Binding.ValidationRules>
                    <ExceptionValidationRule />
                    <DataErrorValidationRule />
                </Binding.ValidationRules>
            </Binding>
        </TextBox.Text>
    </TextBox>
    <TextBox Name="txtEdad" DataContext="{StaticResource Datos}"
        Style="{StaticResource EstiloError}" Width="100">
        <TextBox.Text>
            <Binding Path="Edad" UpdateSourceTrigger="PropertyChanged"
                ValidatesOnExceptions="true" ValidatesOnDataErrors="true" />
        </TextBox.Text>
    </TextBox>
    <Button Height="20" Width="100" Content="Aceptar"></Button>
</StackPanel>

```



ValidatesOnExceptions indica si debe incluirse o no *ExceptionValidationRule*. *ValidatesOnDataErrors* indica si debe incluirse o no *DataErrorValidationRule*, por lo tanto el código usado en ambos TextBox es idéntico, con diferente sintaxis.

ExceptionValidationRule

Representa la regla que verifica las excepciones que son disparadas durante las modificaciones de las propiedades enlazadas.

DataErrorValidationRule

Representa la regla que verifica los errores disparados por la implementación de *IDataErrorInfo*.